banana-brain: training of the brain network, choices and what is left

Date: October 24, 2026 · **Sources**: this session, `audit/2026-10-24-brain-training.pdf` (the measures), `src/brain/network/RawBrainNetwork.cpp` , `lab/src/`

> **How to read this document**
>
> 1. What was chosen and done, with its measured effect.
> 2. What was proposed and not done, or removed, and why.
> 3. What could still be done, ranked by impact and effort.
> 4. What still costs performance in the library.
> 5. Which ideas proved good, which ones did not.
>
> Scope: the training part only (plateaus, jumps, frozen networks, large networks, options, caps, lab defaults). Every number comes from the measures of the previous report (JavaScript simulation of the C++ rules); the performance part comes from reading the C++ code, **not measured** in C++.

Contents

1. Summary
2. Done
3. Not done or removed
4. What could be done
5. Performance left
6. Good and bad ideas
7. Defaults of the lab
8. After the questions: Adam, revive, train()

1. Summary

Point	Conclusion
XOR	16 % solved before, **50 %** with the best training (momentum 0.9, rate 1, path at the init, filter on every input, low new links)
Adam	the best training measured: XOR 96 - 100 %, AND 100 %, **CIRCLE 0 % -> 84 %** (88 % with `revive`), no frozen network; in the lib (`adam()`) and the lab defaults
Lab at the opening	caps of the library + Adam 0.01: XOR, AND and CIRCLE solved (9 seeds of 9 checked in Firefox)
Large networks	AND at 240 hidden: 10 % -> **73 %**; the cause was the path of the second input
Frozen output	understood (unbounded weights, saturated sigmoid); Adam removes almost all of them, `revive()` the last ones
`train()`	now sends the signed error times the rate (a score for the brain): the output moves towards the target from both sides
Caps	new defaults applied: links `3 sqrt n` , steps `4 log2 n` , both at 100 %
Still weak	the curves (SIN4: 0 - 7 % solved, but no freeze with Adam) and the large XOR
Performance	the new options run on the host (every sample reads the GPU memories back); `leftToRight` keeps `n²` ids; the init sorts every neuron against all the others

2. Done

Choice	Where	Measured effect	Verdict
Rate 3 by default (instead of 10)	lab	16 % -> 20 - 31 % XOR, jumps 0.39 -> 0.16	good

Choice	Where	Measured effect	Verdict
Mini-batch (`batch()` , `applyBatch()`)	lab + lib	28 % XOR alone, 40 - 47 % combined, no zigzag	good
Momentum (`momentum()`)	lab + lib	29 % alone at 0.9 / rate 1, 49 - 50 % combined	good on small tasks, bad online on CIRCLE (many jumps)
Rate decay	lab only	15 % (no gain)	not useful
Step limit (`stepLimit()`)	lab + lib	25 % at 0.3, 17 % at 0.1 (too slow)	good at 0.3, against the jumps
Low new links (`lowNewLinks()`)	lab + lib	fewer jumps (0.14), same success	neutral, keeps the curves smooth
Filter on every input	lab (regenerate, batch)	100 % of the networks keep both inputs, error 0.31 -> 0.24	good
Far links (`farLinks()`)	lab + lib	no gain, worse at 120 - 240 hidden	not useful
Path at the init (`pathInit()`)	lab + lib	25 % alone, part of every best combination	good
Keep the paths (`keepPaths()`)	lab + lib	24 % of the networks lost an input path before	good
Score straight on the output (`directOutput()`)	lab + lib	43 % XOR at rate 1 - 3; rescues a frozen output	good as a rescue, unstable with a high rate
Clamp at `weightMax` (`clampWeights()`)	lab + lib	no death, but too little room (CIRCLE 0.31)	not enough alone
Left to right links (`leftToRight()`)	lab + lib	no loop, the bench default	good for feed-forward tasks
Caps `3 sqrt n` / `4 log2 n`	lib defaults	60 hidden: AND 80 %, XOR 70 %; 120 hidden: AND 95 %	good

3. Not done or removed

Idea	Status	Why
Automatic restart of a stuck network	not chosen	the learning is all or nothing: a restart would raise the rate of success without fixing the cause
Weight decay	removed from the lab	applied per sample it pulled every weight to 0: the output became a constant (0.106) on every network
Init scaled by `sqrt(6 / cap)`	removed (your choice)	AND 95 % -> 100 % but XOR 70 % -> 55 %
Caps `2 log2 n + 2`	replaced	too few links on large networks (21 at 1000 neurons)
GPU kernels of the new options	not done	the options run on the host: correct (tested equal) but slow on a large network
A rescue of the saturated hidden neurons	not done	`directOutput` only helps the outputs; a hidden neuron at 0 or 1 still has no gradient

4. What could be done

#	Idea	Impact	Effort	Why
1	GPU kernels for momentum, step limit, clamp, direct output and mini-batch	high on large networks	medium	removes the read back of every memory at each sample
2	Nearest lists computed once for every mode (like `leftToRight`), kept to a few caps	high at the init and the relink	low	the init and each new link sort all the neurons
3	Reduced copy rebuilt only when it is read	medium	low	it is rebuilt after every backpropagation today
4	Mini-batch in `train()`	medium	low	`train` sends the signed error now, one sample at a time
5	Curves (SIN4): more neurons / steps, or a linear output	medium	low	the only task family still under 10 %

#	Idea	Impact	Effort	Why
6	Specialized blocks (lobes) with their own limits	idea	high	see the other report: zones linked weakly to each other
7	Neurons that are born and die (kept for later)	idea	high	the network would also learn its size: a new neuron where the links are full and strong
8	Neurons that move (kept for later)	idea	high	strongly linked neurons get closer: the useful groups form by themselves
9	3D space (kept for later)	idea	medium	more neighbours, shorter paths for the same number of neurons
10	C++ benchmark of the costs of part 5 (planned)	decides the optimizations	low	the costs of part 5 are read from the code, not measured

5. Performance left

Read from the code (not measured in C++). `n` = neurons, `c` = links cap, `s` = steps, `i` = inputs, `o` = outputs.

Where	Cost	When	Problem
Training options on a GPU network	2 copies of `n c` links + `n` neurons per sample	momentum, step limit, clamp, direct output or batch set	the GPU path is skipped: every sample reads the memories back and stores them again
Reduced copy (`sync_`)	`n c` per sample	after every backpropagation	allocated and filled again even when nothing reads it
Read back at a limit	`n c` per sample	a link reaches `weightMin` / `weightMax`	frequent with close limits: the GPU result is copied to the host to relink
Init (`init_`)	`n² log n`	each init / size change / cap change	every neuron sorts all the others (`tools::sort` with `n - 1`)
New link (`relink_`)	`n log n` per strong link	each link at `weightMax` with a free slot	a full sort of the neurons each time (random directions)
`leftToRight` lists (`_left`)	`n²` ids of 8 bytes	kept in memory	10 000 neurons -> about 400 MB; 100 000 -> about 40 GB
`keepPaths`	`2 i (n c)` per removed link	each link at `weightMin`	a path search from every input before and after each removal
`pathInit`	`h n²` per missing pair	the init	fine up to a few thousand neurons, slow beyond
Execution with the caps `3 sqrt n`	`n^1.5 s`	each apply	1000 neurons: about 3.7 M products per apply; 100 000: about 6 G

> **Key takeaway**
> The two first fixes to make are the GPU kernels of the options and the nearest lists computed once and kept short (a few caps): they remove the costs paid at every sample or every init.

6. Good and bad ideas

Idea	Verdict	Evidence
Online training at rate 10	**bad**	zigzag, jumps, 16 % XOR
Every neuron linked to every other one (cap n - 1)	**bad above ~30 neurons**	80 - 85 % frozen at 60 hidden, too slow at 120
Only 1 link per neuron on small networks (old 0.1)	**bad**	0 % everywhere under 20 neurons
Momentum + small rate on small tasks	**good**	XOR 49 - 50 %
Momentum online on many samples	**bad**	CIRCLE: 13 jumps per network, 27 % frozen
Checking every input, not the nearest one	**good**	the main cause of the fall on large networks

Idea	Verdict	Evidence
More links with the size of the network (`3 sqrt n`)	**good**	AND 95 % at 120 hidden
Far random links	**not useful**	no gain at any size
Capping each weight	**not enough**	the sum of many capped weights still saturates
`directOutput` with a high rate	**bad**	mini-batch rate 10 + momentum: 14 % XOR, 160 jumps on CIRCLE

7. Defaults of the lab

Measured to choose what the lab opens with (13 hidden neurons, 600 rounds, 50 networks; base: momentum 0.9, rate 1, path at the init, filter on every input, low new links, keepPaths, left to right).

Variant	XOR solved	AND solved	XOR median round
caps 6 / 8 (lab now)	44 %	88 %	77
caps of the library (12 / 16)	76 %	94 %	41
library caps, random directions	98 %	92 %	11
library caps, orphans off	78 %	94 %	41
library caps, step limit 0.3	74 %	96 %	35
library caps, rate 3 no momentum	66 %	90 %	191
library caps, weightMax 2 (library)	74 %	98 %	28

Then Adam replaced the momentum (part 8). **Applied** (the lab opens with them, the Defaults button comes back to them): caps of the library (`3 sqrt n` links, `4 log2 n` steps: 12 and 16 at 16 neurons), random directions, **Adam 0.01**, path at the init, `keepPaths` , filter on every input, low new links, `allowOrphans` , `weightMax` 5. Checked in Firefox: 3 random seeds of each of XOR, AND and CIRCLE, 9 of 9 solved in 400 rounds.

8. After the questions: Adam, revive, train()

Settings (lab defaults as base)	XOR	AND	CIRCLE	SIN4
momentum 0.9 (former defaults)	98 %	92 %	0 % (92 % frozen)	0 % (10 % frozen)
Adam 0.01	96 %	100 %	84 %	0 % (0 % frozen)
Adam 0.02	100 %	100 %	72 %	0 % (0 % frozen)
Adam 0.05	100 %	—	—	3 %
momentum 0.9 + revive	98 %	—	4 % (4 % frozen, 32 jumps)	0 % (3 % frozen)
Adam 0.01 + revive	—	—	88 % (0 % frozen)	—
mini-batch + Adam 0.02	100 %	—	—	7 %

XOR / AND: 50 networks, CIRCLE: 25, SIN4: 30, 600 rounds; "—" = not measured (the run was stopped).

1. **Adam** goes to the lib (`adam(rate)` , replaces the momentum, saved in the `.rbn`): its first step is the rate whatever the score (1000 or 0.001: tested).
2. **`revive()`** goes to the lib too: alone it unfreezes without learning (CIRCLE 4 %, many jumps), on top of Adam it removes the last freezes (84 -> 88 %).
3. **`train()`** sends `rate x (output - expected)` to a layered network and `rate x (expected - output)` to a brain one; the loss only measures and stops; the `ErrorLossOnBrain` warning is gone.

> **Key takeaway**
> Adam is the change that made the 2D task work (CIRCLE 0 % -> 84 %); the curves are what is left.